

BÀI 1: Giới thiệu về DevOps

Trần Hoàng Sơn

1.1 Thách thức khi thêm tính năng mới

Ngày nay, các công ty đang ngày càng trở nên linh hoạt và dễ thích ứng hơn trong việc tích hợp các nhu cầu của khách hàng để tăng doanh thu. Hãy cùng tìm hiểu quá trình phát triển một sản phẩm hoặc tính năng mới diễn ra như thế nào. Một nhóm phát triển tạo ra mã nguồn, sau đó được nhóm kiểm thử (testing team) kiểm tra, và cuối cùng được triển khai và duy trì bởi nhóm vận hành (operations team). Như bạn có thể thấy, có rất nhiều bước và các nhóm khác nhau tham gia. Với sự phức tạp như vậy, nhiều vấn đề có thể xảy ra. Ví dụ, thường có những tình huống trong ứng dụng hoặc trang web mà một cái gì đó không tải được hoặc có một số lỗi hay thời gian chết. Nhưng những vấn đề này hiếm khi xảy ra với các sản phẩm của Google hoặc Amazon. Các công ty này làm gì khác biệt? Chà, trong số nhiều thứ khác, họ tuân theo nguyên tắc DevOps làm cốt lõi.

Việc thêm một tính năng (mã nguồn) mới vào một máy chủ tập trung hiện có không phải là điều dễ dàng. Sơ đồ dưới đây cho thấy kiến trúc phụ trợ (backend) của một ứng dụng bất kỳ trông như thế nào.

(Mô tả sơ đồ: Sơ đồ cho thấy người dùng (Users) tương tác với các giao diện người dùng (User Interfaces) khác nhau như máy tính để bàn, điện thoại. Các giao diện này kết nối đến một bộ máy xử lý trung tâm (Centralised Engine) trên đám mây. Các nhà phát triển (Developer) cũng tương tác với bộ máy trung tâm này, và bộ máy này lại kết nối với một cơ sở dữ liệu (Database).)

Khi mã nguồn chạy cục bộ mà không có lỗi, nó sẽ được tích hợp vào bộ máy xử lý trung tâm. Giờ đây, ứng dụng có thể được truy cập từ các thiết bị khác nhau như máy tính xách tay, điện thoại thông minh và máy tính bảng. Do đó, nó cần phải tương thích với tất cả các thiết bị đó.

Cũng có một số tắc nghẽn có thể phát sinh khi thêm các tính năng mới:

- Xác suất lỗi cao
- Các thay đổi khó khăn và tốn kém
- Khó khăn trong việc quay trở lại các phiên bản trước

1.2 Tìm hiểu về DevOps

- **DevOps** có thể được định nghĩa là một văn hóa hoặc một triết lý nhằm mục đích tăng cường sự hợp tác giữa nhóm phát triển (Dev) và nhóm vận hành (Ops), dẫn đến việc tạo ra phần mềm chất lượng tốt hơn.
- Trước đây, nhóm Dev và Ops làm việc trong các "silo" (đơn vị biệt lập). Giờ đây trong DevOps, họ làm việc cùng nhau như một nhóm kỹ thuật duy nhất.
- Sự hợp tác gia tăng và tầm nhìn thống nhất duy nhất này giữa các nhóm khác nhau đóng một vai trò quan trọng trong sự phát triển của nhiều tổ chức.
- Ví dụ, **Netflix** chỉ có 8,4 triệu người đăng ký vào năm 2008, nhưng khi công ty chuyển sang đám mây và mở rộng quy mô hệ thống của mình theo các phương pháp DevOps, họ đã tăng trưởng vượt bậc. Đến năm 2015, số lượng người đăng ký đã tăng gấp tám lần so với năm 2008.

- Ngày nay, Netflix là nền tảng phát trực tuyến lớn nhất thế giới hoạt động tại 190 quốc gia với khoảng 222 triệu người đăng ký trên toàn cầu.

Các giai đoạn khác nhau có trong chu trình DevOps như sau:

- **Plan (Lập kế hoạch):** Trong giai đoạn này, các nhóm xác định các yêu cầu kinh doanh, thảo luận và tạo ra một lộ trình dự án. Họ cũng sử dụng các công cụ quản lý dự án như bảng Kanban. Ví dụ, trong khi tạo một tính năng, nhóm phát triển có thể phải tương tác với nhóm an ninh mạng để đảm bảo không có lỗ hổng nào được tạo ra trong khi thêm tính năng này.
- **Code (Viết mã):** Trong giai đoạn này, các nhà phát triển có thể ở các vị trí địa lý khác nhau viết mã. Ví dụ, hai nhà phát triển có thể làm việc trên hai thành phần khác nhau của một tính năng trong khi ở các múi giờ khác nhau.
- **Build (Xây dựng):** Trong giai đoạn này, các nhà phát triển đẩy mã nguồn của họ vào kho mã nguồn như GitHub sau khi đảm bảo nó chạy như mong đợi.
- **Test (Kiểm thử):** Trong giai đoạn này, tính năng đã phát triển được kiểm tra kỹ lưỡng với các trường hợp sử dụng thử nghiệm khác nhau.
- **Release (Phát hành):** Trong giai đoạn này, toàn bộ nhóm kỹ thuật làm việc để lên lịch phát hành và tạo các gói phát hành (đóng gói tất cả các thư viện và phần phụ thuộc bên ngoài được yêu cầu bởi một ứng dụng).
- **Deploy (Triển khai):** Trong giai đoạn này, gói phát hành được triển khai vào môi trường sản xuất, nơi người dùng cuối có thể truy cập được.
- **Operate (Vận hành):** Trong giai đoạn này, nhóm kỹ thuật quản lý việc cung cấp (thiết lập) và cấu hình cơ sở hạ tầng.
- **Monitor (Giám sát):** Trong giai đoạn này, các chỉ số hiệu suất khác nhau, chẳng hạn như số lượng yêu cầu và thời gian phản hồi của máy chủ, cùng với những thứ khác, được giám sát.

(Mô tả sơ đồ: Sơ đồ vòng lặp DevOps hiển thị các giai đoạn: Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor, và quay trở lại Plan. Xung quanh vòng lặp là logo của các công cụ phổ biến được sử dụng trong từng giai đoạn như Jira, Git, Maven, Selenium, Jenkins, Docker, Ansible, Kubernetes, Nagios, Splunk.)

Chu trình DevOps hoàn chỉnh này có thể được chia thành hai phần:

- **Continuous Integration (CI - Tích hợp liên tục):** Bao gồm các giai đoạn plan, code, build và test. Trong bốn giai đoạn này, mã được viết bởi các nhà phát triển được đẩy liên tục, điều này kích hoạt các trường hợp kiểm thử tự động chạy. Nếu mã thất bại, các nhà phát triển sẽ được thông báo ngay lập tức; nếu không, mã sẽ chuyển sang bước tiếp theo.
- **Continuous Delivery (CD - Phân phối liên tục):** Bao gồm các giai đoạn release, deploy, operate và monitor. Sau khi mã được kiểm tra đúng cách, nó sẽ được chuyển đổi thành các gói phát hành và triển khai vào môi trường sản xuất, nơi ứng dụng được giám sát liên tục.

1.3 Các nguyên tắc của DevOps

Bất cứ khi nào một tổ chức bắt đầu triển khai các phương pháp DevOps, họ đều tìm kiếm các nguyên tắc cơ bản sau:

(Mô tả sơ đồ: Sơ đồ gồm 4 ô xung quanh một hình bộ não ở trung tâm, đại diện cho 4 nguyên tắc: Collaboration (Hợp tác), Customer Centricity (Lấy khách hàng làm trung tâm), Automation (Tự động hóa), và Continuous Improvement (Cải tiến liên tục).)

- **Collaboration (Hợp tác):** DevOps tăng cường sự hợp tác giữa các nhóm khác nhau làm việc trên một tính năng. Các nhóm phát triển và vận hành làm việc cùng nhau, chia sẻ phản hồi liên tục và giao tiếp trong suốt chu trình phát triển. Do đó, các sản phẩm chất lượng cao được tạo ra.
- **Customer-centricity (Lấy khách hàng làm trung tâm):** Trong DevOps, việc phát triển diễn ra trong các chu kỳ ngắn. Các nhóm thu thập phản hồi liên tục từ người dùng cuối và thực hiện các cải tiến trong sản phẩm. Phản hồi liên tục từ người dùng cuối cũng giúp chúng ta phân tích liệu tác động mong đợi có được tạo ra hay không.
- **Continuous Improvement (Cải tiến liên tục):** Vì DevOps hỗ trợ triển khai nhanh chóng và phản hồi liên tục, ứng dụng được phát triển sẽ liên tục được cải tiến.
- **Automation (Tự động hóa):** Nhu cầu của khách hàng liên tục thay đổi và các nhóm triển khai mã của họ rất thường xuyên; do đó, việc kiểm tra lỗi thủ công, tạo gói triển khai, v.v. có thể được thực hiện một cách tự động đến mức bất cứ khi nào một nhà phát triển đẩy mã, các bước xây dựng và kiểm thử sẽ được kích hoạt tự động. Nếu phần mềm vượt qua tất cả các bài kiểm tra, nó sẽ được triển khai vào sản xuất. Ngoài ra, nếu mã thất bại, các nhà phát triển sẽ được thông báo ngay lập tức để họ có thể kiểm tra bản dựng không thành công. Vì vậy, lợi thế của tự động hóa là các nhà phát triển có thể tập trung vào việc viết mã hơn là lo lắng về các bước khác.

1.4 Phiên bản ngữ nghĩa (Semantic Versioning)

Với sự ra đời của DevOps, tần suất các nhà phát triển triển khai mã đã tăng lên và với phản hồi liên tục từ người dùng cuối, các ứng dụng đã được cải tiến liên tục. Với mỗi bản phát hành mới, sửa lỗi hoặc sửa đổi trong mã hiện có, một số duy nhất được gán cho bản phát hành mới. Quá trình duy trì các phiên bản (bản phát hành) khác nhau của một phần mềm cụ thể được gọi là phiên bản phần mềm và phương pháp mà bản phát hành được đánh số được gọi là phiên bản ngữ nghĩa. Phiên bản ngữ nghĩa sử dụng ba số để biểu thị các phiên bản phần mềm khác nhau theo cấu trúc X.Y.Z (Major.Minor.Patch), như trong hình dưới đây.

(Mô tả sơ đồ: Sơ đồ giải thích cấu trúc phiên bản X.Y.Z. Major Version (X) được tăng lên khi có thay đổi lớn như thay đổi kiến trúc. Minor Version (Y) được tăng lên khi có thay đổi nhỏ không phá vỡ API, chẳng hạn như thêm tính năng mới. Patch Version (Z) được tăng lên khi có sửa lỗi.)

Bây giờ, hãy hiểu ý nghĩa của từng thành phần:

- **Major (X):** Số ngoài cùng bên trái chỉ được tăng lên khi có một thay đổi lớn diễn ra, chẳng hạn như thay đổi kiến trúc. Nếu phiên bản chính được tăng lên, thì phiên bản phụ và phiên bản vá lỗi sẽ được đặt lại về 0.
- **Minor (Y):** Số ở giữa chỉ được tăng lên khi có một thay đổi nhỏ xảy ra, chẳng hạn như thêm một tính năng mới. Nếu phiên bản phụ được tăng lên, phiên bản vá lỗi sẽ được đặt lại về 0.
- **Patch (Z):** Số ngoài cùng bên phải được tăng lên trong trường hợp có bất kỳ lỗi nào hoặc vấn đề nào được khắc phục.

1.5 Các môi trường khác nhau được sử dụng trong kiểm thử phần mềm

Một đoạn mã không bao giờ đi thẳng từ môi trường phát triển đến môi trường sản xuất. Đầu tiên, nó đi qua môi trường phát triển, nơi các nhà phát triển viết mã, đến môi trường kiểm thử, nơi quá trình kiểm thử diễn ra. Sau đó, khi mã vượt qua môi trường kiểm thử, nó sẽ chuyển đến môi trường dàn dựng (staging). Một môi trường dàn dựng tương tự như một môi trường sản xuất. Nó giúp chúng ta hiểu một ứng dụng sẽ hoạt động như thế nào trong một môi trường sản xuất thực tế. Một nhóm mẫu người dùng sử dụng ứng dụng trong

môi trường dàn dựng và cung cấp phản hồi. Sau đó, ứng dụng sẽ được chuyển đến môi trường sản xuất nếu phản hồi đáp ứng các tiêu chí.

*(Mô tả sơ đồ: Sơ đồ so sánh hai luồng. Luồng sai (đánh dấu X) chỉ có Development -> Production. Luồng đúng (đánh dấu tick) có Development -> Testing -> Staging -> Production.)** Staging -> Production.* Production -> Testing -> Staging -> Production.)*

Session 2: Các Nguyên Tắc Viết Mã Sạch

Tác giả: Trần Hoàng Sơn

2.1 Tầm Quan Trọng Của Việc Viết Mã Sạch

DevOps loại bỏ sự tách biệt giữa các nhóm khác nhau, chẳng hạn như phát triển, vận hành và QA, dẫn đến việc phát triển phần mềm chất lượng cao. Các nhà phát triển viết và triển khai mã của họ thường xuyên hơn trước đây. Nhưng với sự linh hoạt như vậy, điều cần thiết là phải đảm bảo rằng mã được viết có điểm chất lượng tốt về mặt thiết kế. Do đó, các nhà phát triển nên tuân theo các nguyên tắc viết mã sạch nhất định bất cứ khi nào họ viết mã. Những ưu điểm của việc tuân theo các nguyên tắc viết mã sạch như sau:

- Mã sạch dễ hiểu, sửa đổi và bảo trì trong dài hạn.
- Bất cứ khi nào một nhà phát triển viết mã sạch, các nhà phát triển khác sẽ dễ dàng làm việc với nó, nếu được yêu cầu, trong tương lai.
- Viết mã sạch đóng vai trò là nền tảng cho toàn bộ dự án trong dài hạn, tương tự như một tòa nhà có nền móng vững chắc để nó tồn tại lâu dài.

Bây giờ, giả sử một nhà phát triển viết một đoạn mã với mục đích làm cho chức năng được yêu cầu hoạt động và đáp ứng thời hạn mà không tuân theo bất kỳ nguyên tắc tiêu chuẩn nào về viết mã. Trong trường hợp đó, mã sẽ hoạt động ban đầu nhưng có thể gây ra sự cố trong dài hạn vì những lý do sau:

- Giả sử, trong trường hợp không có nhà phát triển ban đầu làm việc trên mã; một nhà phát triển mới làm việc trên đó. Để nhà phát triển mới có thể sửa lỗi hoặc làm việc trên cùng một mã, tức là, để làm việc hiệu quả, sẽ mất rất nhiều thời gian vì mã sẽ rất phức tạp và khó hiểu đối với họ.
- Với mỗi chu kỳ (sprint), việc kiểm thử trở nên rất khó khăn và chi phí để thêm một tính năng mới tăng lên. Ví dụ, việc thêm một tính năng đơn giản, tầm thường cũng mất thời gian.
- Nhiều vấn đề thiết kế có thể liên quan đến mã, chẳng hạn như các thành phần được ghép nối chặt chẽ, khả năng mở rộng, khả năng tái sử dụng, các biến và tên

lớp không phù hợp.

Do đó, việc không giải quyết các vấn đề như vậy cuối cùng sẽ dẫn đến một cơ sở mã lộn xộn, trở nên rất khó quản lý theo thời gian và có thể dẫn đến thời gian chết của ứng dụng. Do đó, trong những trường hợp xấu nhất, một tổ chức cũng có thể cần phải bắt đầu lại mọi thứ từ đầu.

2.2 Tìm Hiểu Các Nguyên Tắc Viết Mã Sạch

Các nguyên tắc khác nhau để viết mã sạch như sau:

- **Quy ước đặt tên có ý nghĩa:** Tên của biến phải luôn mang tính mô tả và làm nổi bật mục đích của nó.
- **Mô-đun hóa mã:** Điều này đề cập đến quá trình tổ chức mã thành các hàm để hàm có thể được tái sử dụng và mã trông ngắn gọn và dễ đọc. Nó còn được gọi là nguyên tắc ‘không lặp lại chính mình’ (DRY).
- **Tối ưu hóa mã:** Tối ưu hóa mã liên quan đến các khía cạnh thực thi thời gian chạy và sử dụng bộ nhớ của một chương trình.
- **Tài liệu hóa mã:** Tài liệu hóa mã đảm bảo rằng mã được ghi chép đầy đủ và làm nổi bật các chi tiết của từng khía cạnh của mã, chẳng hạn như các đối số của hàm, kiểu trả về và mô tả.
- **Linting mã:** Linting mã là quá trình kiểm tra xem các nguyên tắc liên quan đến ngôn ngữ lập trình có được tuân thủ hay không. Ví dụ, Python tuân theo các nguyên tắc được mô tả trong PEP8 (Đề xuất cải tiến Python).

2.2.1 Tái Cấu Trúc Mã

Thông thường, các nhóm phát triển thỏa hiệp về chất lượng của mã để phát hành một tính năng nhanh chóng trên thị trường và đáp ứng nhu cầu của khách hàng. Họ có thể hoàn thành nhiệm vụ của mình và giành được sự tin tưởng của khách hàng nhưng phải đánh đổi một số chất lượng. Đây được gọi là ‘nợ kỹ thuật’ và cần được tổ chức xử lý trong tương lai để duy trì mã trong dài hạn. Nếu nợ kỹ thuật tiếp tục tích lũy, việc thêm bất kỳ tính năng mới nào cũng có thể trở nên gần như không thể.

Một giải pháp cho vấn đề này là tái cấu trúc mã. Đó là quá trình cải thiện cấu trúc bên trong của mã để dễ đọc và sửa đổi mà không làm thay đổi hành vi bên ngoài (chức năng) của nó.

Trong Python, có nhiều cách khác nhau để tái cấu trúc mã của bạn và làm cho nó ngắn gọn và dễ đọc hơn, một số trong đó như sau:

- **Sử dụng list comprehensions:** Đây là một trong những cách thanh lịch hoặc Pythonic nhất để tạo một danh sách dựa trên các danh sách hiện có. Mã được viết theo cách này ngắn gọn, sạch sẽ và dễ đọc. Cú pháp cơ bản của list comprehension trông như thế này.

```
[<biểu thức> <for mục trong danh sách> <if điều kiện>]
```

Trường hợp 1: List Comprehensions thay vì vòng lặp for

```
# Mã để tạo squared_list mới từ các số (nếu lẻ)
my_numbers = [1,2,3,4,5]

sqr_list = []
for item in my_numbers:
    if item % 2 == 1:
        sqr_list.append(item ** 2)

# Thay thế bằng một dòng duy nhất
sqr_list = [value ** 2 for value in my_numbers if value % 2 == 1]
print(sqr_list)
```

- **Gán bằng cách giải nén tuple:** Không giống như khai báo và khởi tạo từng biến trên một dòng riêng biệt, Python hỗ trợ một cách thành ngữ hơn để đạt được điều này được gọi là giải nén tuple.

Trường hợp 2: Gán giá trị sử dụng tuple unpacking

```
name = 'Steve'
age = 44
roll_no = 28

# Có thể thực hiện bằng tuple unpacking trên một dòng
name, age, roll_no = 'Steve', 44, 28
```

- **Sử dụng các hàm any và all:** Bạn cũng có thể sử dụng `any` và `all`, hai hàm tích hợp sẵn của Python để tái cấu trúc mã của mình. Hàm `any` kiểm tra iterable (danh sách, tuple, v.v.) được truyền và trả về true nếu bất kỳ phần tử nào hoặc

điều kiện đã cho cho bất kỳ phần tử nào là đúng. Ví dụ, giả sử bạn muốn kiểm tra xem có số chẵn nào trong `num_list = [12, 22, 33, 44, 55]` hay không. Sau đó, mã được viết bằng vòng lặp `for` có thể được thay thế bằng hàm `any` như hình dưới đây.

Trường hợp 3: Sử dụng hàm `any` và `all`

```
# Mã cũ để kiểm tra xem có số chẵn nào không
num_list = [12, 22, 33, 44, 55]

for value in num_list:
    if value % 2 == 0:
        is_even_present = True
        break

# Sử dụng hàm any
is_even_present = any(value % 2 == 0 for value in num_list)
```

Tương tự, hàm `all` kiểm tra xem điều kiện có đúng với mọi giá trị hay không. Ngay khi nó thấy rằng điều kiện không đúng với bất kỳ giá trị nào, nó sẽ trả về `false`; nếu không, nó sẽ trả về `true` sau khi chạy qua toàn bộ danh sách.

- **Sử dụng các hàm `lambda`:** Các hàm `lambda` cũng có thể được sử dụng để tái cấu trúc mã trong Python. Thông thường, một hàm có một biểu thức duy nhất. Trong những trường hợp như vậy, một hàm ẩn danh (không tên) đơn giản được gọi là hàm `lambda` có thể được định nghĩa. Cú pháp chung của các hàm `lambda` như sau.

```
lambda <danh sách đối số>: biểu thức
```


Trường hợp 4: Hàm Lambda

```
# Một hàm đơn giản để tìm số lớn hơn
def get_greater_number(num_1, num_2):
    if num_1 > num_2:
        return num_1
    return num_2

# Sử dụng lambda (hàm ẩn danh) trong một dòng
get_greater = lambda num_1, num_2 : num_1 if num_1 > num_2 else num_2
print(get_greater(12, 22))
```

2.2.2 Tối Ưu Hóa Mã

Tối ưu hóa mã của bạn về độ phức tạp về thời gian và không gian là rất quan trọng. Python là một trong những ngôn ngữ lập trình phổ biến nhất trong khoa học dữ liệu, học máy, học sâu và AI vì nó dễ viết mã và thực hiện các tác vụ. Tuy nhiên, các vấn đề trở nên rõ ràng khi bạn triển khai ứng dụng của mình trên quy mô lớn. Ví dụ, bạn đã phát triển một ứng dụng học tập điện tử và triển khai nó trong sản xuất. Ban đầu, nó hoạt động tốt, nhưng hiệu suất của nó giảm dần khi số lượng người dùng tăng lên và ứng dụng của bạn được mở rộng quy mô. Đây là lúc tối ưu hóa mã là quan trọng. Nó có thể cải thiện thời gian thực thi mã hoặc việc sử dụng bộ nhớ hoặc cả hai.

Độ phức tạp về thời gian

Một nhà phát triển có thể tối ưu hóa thời gian thực thi mã của họ theo nhiều cách. Đó là:

- Sử dụng list comprehensions thay cho vòng lặp for
- Sử dụng các hàm tích hợp sẵn như `sum`, `map` và `reduce` bất cứ khi nào có thể. Các hàm này đã được định nghĩa trong C và được tối ưu hóa. Ví dụ, để cộng các số có trong danh sách chẳng hạn như `num_list = [1, 2, 3, 5, 6]`, bạn sẽ làm như sau.

```
result = 0
for value in num_list:
    result += value
```

Bạn có thể thực hiện phép cộng này bằng cách sử dụng các hàm tích hợp sẵn bằng cách chỉ cần gọi hàm `sum` trên mảng; ví dụ, `result = sum(num_list)`.

- Các biến cục bộ nên được ưu tiên hơn các biến toàn cục vì các lỗi hổng bảo mật liên quan đến các biến toàn cục.

Độ phức tạp về không gian

Ngoài ra, các nhà phát triển cũng có thể tối ưu hóa bộ nhớ theo nhiều cách khác nhau.

- Ví dụ, sử dụng list comprehensions làm cho mã nhanh, nhưng thiếu sót là chúng tạo ra một danh sách mới trong mọi trường hợp. Vì vậy, nếu bạn muốn xóa các phần tử khỏi một danh sách lớn bằng cách sử dụng list comprehension, nó sẽ không phù hợp vì nó sẽ yêu cầu bộ nhớ lớn để xóa các phần tử so với vòng lặp `'for'`. Sử dụng vòng lặp `'for'`, bạn có thể xóa các phần tử bằng cách sử dụng các hàm `list.pop()` hoặc `list.remove()` trong danh sách ban đầu thay vì tạo một danh sách mới.
- Tương tự, nhiều khi, việc sử dụng một mảng không phù hợp vì các giá trị được lưu trữ một cách liên kề và có thể một lượng bộ nhớ liên kề cao như vậy không có sẵn. Do đó, danh sách liên kết rất hữu ích trong những trường hợp như vậy. Tuy nhiên, việc tìm kiếm các phần tử trong danh sách liên kết chậm hơn so với một mảng. Do đó, luôn có một sự đánh đổi nhất định giữa việc chọn một mảng và một danh sách liên kết về việc sử dụng bộ nhớ.

2.2.3 Tài Liệu Hóa Mã

Nhiều nhà phát triển cho rằng mã sạch tự nó đã nói lên điều đó. Ví dụ, nếu chúng ta đã đặt cho các biến, lớp và hàm của bạn những cái tên có ý nghĩa và mô tả, thì việc hiểu mã sẽ trở nên dễ dàng. Tuy nhiên, khi quy mô của cơ sở mã tăng lên, nó trở nên khó quản lý vì để hiểu một mã cụ thể làm gì, bạn sẽ không đọc lại mã của mình từ đầu. Đây

là lúc tài liệu hóa mã phục vụ mục đích của nó. Nó chứa một mô tả về các phần khác nhau của mã, rất hữu ích cho bạn, các thành viên trong nhóm của bạn và những người mới tham gia, nếu có, trong nhóm.

Docstrings

Trong python, việc ghi lại một mã rất dễ dàng và tiện dụng với docstrings (chuỗi tích hợp sẵn). Các docstrings này xác định hành vi hoàn chỉnh của các đối tượng như lớp, hàm, v.v. Cú pháp chung của việc viết một docstring là giữa ‘dấu ngoặc đơn ba’ hoặc ‘dấu ngoặc kép ba’. Ngoài ra, docstring cho bất kỳ lớp, hàm nào, v.v., cần phải có mặt ngay sau tên của lớp và các hàm. Docstring này được lưu trữ trong thuộc tính `__doc__` của đối tượng, có thể được truy cập bằng `tên_đối_tượng.__doc__`. Docstring cũng có thể được truy cập bằng hàm `help`. Ví dụ, `help(tên_hàm)` cũng sẽ in docstring của hàm hoặc đối tượng được truyền dưới dạng đối số. Hình ảnh dưới đây cho thấy cách một docstring nên được định nghĩa cho một hàm.

```
def get_neighbors(some_list, index):
    """Trả về các phần tử lân cận của chỉ số trong danh sách.

    Parameters:
        some_list (list): Bất kỳ danh sách cơ bản nào.
        index (int): Vị trí của phần tử trong some_list.

    Returns:
        neighbors (list): Danh sách phần tử bên cạnh index.
    """
    neighbors = []
    if index - 1 >= 0:
        neighbors.append(some_list[index-1])
    if index < len(some_list):
        neighbors.append(some_list[index+1])
    return neighbors

print(help(get_neighbors))
```

Bình luận và docstrings

Trong Python, các bình luận cải thiện khả năng đọc và cho biết mã hoạt động như thế nào, trong khi các docstrings cho biết mã làm gì. Nó cung cấp cho chúng ta một mô tả về các hàm, phương thức hoặc lớp. Docstrings có thể được coi là một loại chuỗi hoặc

bình luận nhiều dòng đặc biệt được đặt ngay trong khối định nghĩa của tiêu đề hàm hoặc lớp.

Lưu ý: Mặc dù một docstring và một bình luận nhiều dòng có vẻ tương tự, nhưng chúng khác nhau. Các bình luận nhiều dòng có thể được đưa ra ở bất cứ đâu trong chương trình, nhưng các docstrings chỉ có thể ở đầu một hàm hoặc một lớp.

2.2.4 Linting Mã

Cùng với việc ghi lại mã của bạn, việc đảm bảo rằng mã có chất lượng cao cũng rất cần thiết. Python có một bộ nguyên tắc để đảm bảo rằng chất lượng mã cao và đáp ứng các tiêu chuẩn của ngôn ngữ. Các nguyên tắc này được định nghĩa bởi một tài liệu có tên là PEP8, được viết vào năm 2001. Để kiểm tra xem mã của chúng ta có tuân theo các nguyên tắc này hay không, chúng ta sử dụng các linter. Linter là các công cụ kiểm tra mã của bạn và cố gắng tìm ra sự không nhất quán hoặc các cấu trúc mã không phù hợp. Một vài trong số các linter Python nổi tiếng là pycodestyle, pylint, v.v. Một số nguyên tắc viết mã cần được tuân thủ khi viết mã như sau:

- Thụt lề

E1 Thụt lề (Indentation)

```
# KHÔNG KHUYẾN NGHỊ
def get_uppercase(name):
    print('In upper_case :')
    print(name.upper())
def get_lowercase(name):
    print('In lower_case :')
```



```
# KHUYẾN NGHỊ
def get_uppercase(name):
    print('In upper_case :')
    print(name.upper())

def get_lowercase(name):
    print('In lower_case :')
```

- Khoảng trắng

E2 Khoảng trắng (Whitespace)

```
# KHÔNG KHUYẾN NGHỊ
num_list = [ 12, 22, 33 ]
val_1, val_2 = 2, 3
result = val_1 ** 2 + | val_2

# KHUYẾN NGHỊ
num_list = [12, 22, 33]
val_1, val_2 = 2, 3
result = val_1 ** 2 + val_2
```

- Dòng trống

E3 Dòng trống (Blank line)

```
# KHÔNG KHUYẾN NGHỊ
def get_uppercase(name):
    print('In upper_case :')

    print(name.upper())

# KHUYẾN NGHỊ - chỉ một dòng trống giữa các hàm
def get_uppercase(name):
    print('In upper_case :')
    print(name.upper())
```

- Độ dài dòng

E5 Độ dài dòng (Line length)

```
# KHÔNG KHUYẾN NGHỊ
print('Đây là demo quy tắc độ dài dòng tối đa từ pip8')

# KHUYẾN NGHỊ - Độ dài dòng tối đa <= 79
print('Đây là demo quy tắc độ dài dòng \
tối đa từ pip8')
```

Session 3: Tìm Hiểu Về Kiểm Thử Đơn Vị (Unit Testing)

Tác giả: Trần Hoàng Sơn

3.1 Hiểu Về Kiểm Thử Phần Mềm

Kiểm thử phần mềm là quá trình xác nhận và xác thực xem một phần mềm hoặc ứng dụng có không có lỗi hay không. Điều này cũng bao gồm việc kiểm tra xem tất cả các thông số kỹ thuật được thiết lập trong quá trình thiết kế và phát triển có được thực hiện tốt hay không và nó có đáp ứng hiệu quả các yêu cầu của người dùng trong khi xử lý tất cả các 'trường hợp ngoại lệ' hay không. Kiểm thử chương trình không chỉ cố gắng xác định các sai sót trong phần mềm/mã mà còn xác định các cách để tăng hiệu suất và khả năng sử dụng của phần mềm.

Tuy nhiên, việc kiểm thử cần được thực hiện đúng cách để tránh trải nghiệm người dùng tồi tệ và trong một số trường hợp, cũng có thể làm sập toàn bộ ứng dụng. Ví dụ, bạn có thể đã thấy nhiều trang web của chính phủ bị sập vào ngày công bố kết quả thi. Điều này là do số lượng người dùng vượt quá khả năng của trang web. Hơn nữa, bạn sẽ thường xuyên gặp phải các sự cố kỹ thuật trên một số trang web và ứng dụng ngay sau khi có bản cập nhật phần mềm. Tất cả những vấn đề này có thể bắt nguồn từ việc kiểm thử người dùng không hiệu quả (đây có thể chỉ là một trong những nguyên nhân góp phần trong một số trường hợp). Tuy nhiên, hậu quả của việc kiểm thử không đúng cách không chỉ giới hạn ở việc ảnh hưởng đến trải nghiệm của người dùng cuối.

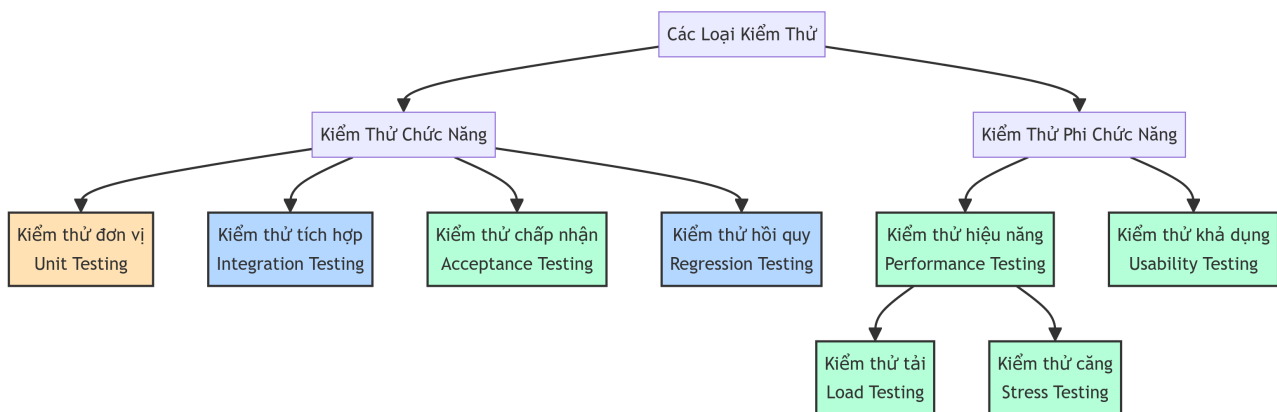
Tầm Quan Trọng Của Testing

- ✓ Ngăn chặn lỗi (Prevents bugs)
- ✓ Giảm chi phí phát triển (Reduces costs)
- ✓ Cải thiện hiệu năng (Improves performance)
- ✓ Nâng cao trải nghiệm người dùng (Enhances UX)

Một số vấn đề mà kiểm thử giải quyết cho chúng ta như sau:

- Ngăn chặn lỗi
- Giảm chi phí phát triển
- Cải thiện hiệu suất
- Nâng cao trải nghiệm người dùng

3.2 Các Loại Kiểm Thử Phần Mềm Khác Nhau



Kiểm thử phần mềm có thể được phân loại rộng rãi thành hai loại:

- **Kiểm thử chức năng (Functional Testing)**
- **Kiểm thử phi chức năng (Non-Functional Testing)**

Kiểm Thử Chức Năng

Kiểm thử chức năng có thể được phân loại thành các loại khác nhau, như sau:

- **Kiểm thử đơn vị (Unit testing):** Được thực hiện bởi các nhà phát triển trong môi trường phát triển để kiểm tra chức năng của các tính năng hoặc chức năng riêng lẻ (nhỏ nhất).
- **Kiểm thử tích hợp (Integration testing):** Được thực hiện để kiểm tra xem các tính năng khác nhau có hoạt động bình thường khi được sử dụng như một thực thể kết hợp hay không. Nó diễn ra trong môi trường kiểm thử.
- **Kiểm thử chấp nhận (Acceptance testing):** Thường liên quan đến việc kiểm tra nhu cầu, yêu cầu của người dùng và các quy trình kinh doanh để xem một hệ thống có đáp ứng các tiêu chí chấp nhận hay không. Ở đây, một nhóm nhỏ người

dùng cuối sử dụng ứng dụng và đưa ra phản hồi của họ về việc nó có hoạt động như mong đợi hay không. Việc kiểm thử này diễn ra trong môi trường dàn dựng (staging).

- **Kiểm thử hồi quy (Regression testing):** Đảm bảo rằng bất kỳ tính năng mới nào được thêm vào hoặc các sửa đổi được thực hiện không gây ra bất kỳ lỗi mới hoặc cũ nào trong ứng dụng hiện có. Việc kiểm thử này diễn ra trong môi trường kiểm thử.

Kiểm Thử Phi Chức Năng

Ngoài việc phục vụ các yêu cầu chức năng, một ứng dụng phải hoạt động tốt hơn về tải, căng thẳng, thời gian phản hồi, v.v., khi được triển khai vào môi trường sản xuất. Các tham số này thuộc kiểm thử phi chức năng. Các loại kiểm thử phi chức năng khác nhau như sau:

- **Kiểm thử hiệu năng (Performance testing):** Có thể được phân loại thành hai loại:
 - **Kiểm thử tải (Load testing):** Được thực hiện để đảm bảo rằng ứng dụng hoạt động tốt với khối lượng công việc dự kiến trong đời thực. Giả sử khối lượng công việc dự kiến cho một trang web là 1 triệu người dùng đồng thời. Trong trường hợp này, chúng tôi sẽ thực hiện các bài kiểm tra tải cho các vị trí địa lý khác nhau, thời gian phiên, v.v. và kiểm tra xem ứng dụng có hoạt động tốt về thời gian phản hồi hay không.
 - **Kiểm thử căng thẳng (Stress testing):** Được thực hiện để tìm ra điểm gãy của ứng dụng. Ở đây, chúng tôi cung cấp một khối lượng công việc cao hơn so với khối lượng công việc dự kiến trong đời thực. Sử dụng kiểm thử căng thẳng giúp chúng tôi đảm bảo rằng ứng dụng hoạt động tốt dưới các khối lượng công việc không mong muốn, kiểm tra xem hệ thống có lưu dữ liệu trước khi gặp sự cố hay không, v.v.
- **Kiểm thử khả dụng (Usability testing):** Được thực hiện để hiểu ứng dụng dễ sử dụng và thân thiện với người dùng như thế nào. Nó diễn ra trong môi trường dàn dựng (staging).

3.3 Kiểm Thử Đơn Vị Với Pytest

Bất cứ khi nào một nhà phát triển viết bất kỳ mã nào, họ cũng phải viết một số bài kiểm thử đơn vị nhất định để đảm bảo rằng các tính năng nhỏ nhất đang hoạt động chính xác và như mong đợi. Python hỗ trợ một mô-đun sẵn có tên là ‘Pytest’ , mà các nhà phát triển có thể sử dụng để chạy và xác minh các trường hợp kiểm thử.

- Pytest có thể được sử dụng để viết cả các trường hợp kiểm thử đơn giản và phức tạp.
- Để cài đặt Pytest, bạn chỉ cần chạy lệnh `pip install pytest` .
- Khi viết các trường hợp kiểm thử, bạn phải sử dụng từ khóa `assert` . Nó là một từ khóa duy nhất được sử dụng để gỡ lỗi mã. Nó kiểm tra xem điều kiện đã cho là đúng hay sai. Trong trường hợp một điều kiện là sai, nó sẽ gây ra lỗi xác nhận (assertion error).
- Pytest chỉ chọn hàm có tên bắt đầu bằng ‘test’ . Trong trường hợp tên của một hàm khác, nó sẽ bị bỏ qua.
- Lệnh được sử dụng để chạy các trường hợp kiểm thử là `pytest -v` , lệnh này hiển thị chi tiết kết quả của các hàm kiểm thử.
- Tên tệp mà bạn đang viết các trường hợp kiểm thử phải ở định dạng ‘test_.py’ hoặc ‘test.py’ . Khi bạn chạy lệnh `pytest -v` , nó sẽ kiểm tra tên của các tệp bắt đầu bằng `test_` hoặc kết thúc bằng `_test` trong thư mục hiện tại. Khi nó tìm thấy các tệp và đi vào bên trong chúng, nó sẽ kiểm tra thêm tên của các hàm và, như đã nêu trước đó, chỉ chọn các hàm có tên bắt đầu bằng ‘test’ .

Hoạt động của Pytest

Hãy lấy ví dụ được đề cập trong tài liệu chính thức để hiểu cách hoạt động của kiểm thử đơn vị.

Tạo test đầu tiên với Pytest

Tạo test đầu tiên của bạn

Tạo file mới tên `test_sample.py`, chứa một hàm và một test:

```
# content of test_sample.py
def func(x):
    return x + 1

def test_answer():
    assert func(3) == 5
```

Test này

```
$ pytest
===== test session starts =====
platform Linux -- Python 3.x.y, pytest-7.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 1 item

test_sample.py F [100%]

===== FAILURES =====
_____ test_answer _____
```

Trong ví dụ, bạn có thể thấy có hai hàm `func(x)` và `test_answer()`. Hàm `func(x)` nhận một đối số nguyên `x`, tăng nó lên 1, rồi trả về kết quả. Bây giờ để kiểm tra hoạt động của hàm này, có một hàm khác tên là `test_answer`. Hàm này sử dụng từ khóa `assert` để so sánh đầu ra của hàm `func(x)` với một giá trị cụ thể. Ví dụ: Trong ảnh chụp màn hình ở trên, bạn có thể thấy rằng `func(3)` được so sánh với 5. Bài kiểm thử này không thành công vì sau khi gọi hàm `func(3)`, giá trị 3 được tăng lên 1 và 4 được trả về. Tuy nhiên, trong từ khóa `assert`, đầu ra mong đợi là 5, điều này không chính xác. Do đó, một thông báo lỗi được hiển thị ở cuối.

3.4 Kiểm Thử Thủ Công so với Kiểm Thử Tự Động

Kiểm thử thủ công dễ bị lỗi do con người, trong khi kiểm thử đơn vị tự động thì không. Bên cạnh đó, kiểm thử tự động cung cấp cho chúng ta dữ liệu định lượng tốt hơn, cho biết một tính năng tổng thể hoạt động tốt như thế nào.

Ngoài ra, bạn có thể chạy một lệnh duy nhất, `pytest -v`, để xác minh tất cả các bài kiểm thử đơn vị cùng một lúc. Bây giờ, nếu bạn phải làm điều tương tự theo cách thủ công, bạn có thể phải chạy từng trường hợp kiểm thử riêng biệt. Trên thực tế, nếu có nhiều tệp kiểm thử, việc kiểm thử thủ công trở nên rất khó khăn. Ngay cả trong tình huống như vậy, `pytest -v` sẽ hoạt động liền mạch.

3.5 Các Phương Pháp Tốt Nhất Để Kiểm Thử

Một số phương pháp tốt nhất cần được xem xét và tuân thủ trong khi thực hiện kiểm thử phần mềm như sau:

- **Tính nguyên tử của mã được kiểm thử (Atomicity):** Điều này đảm bảo rằng mã (dù là một hàm hay một lớp) có một trách nhiệm duy nhất. Nếu một mã cần được kiểm thử có nhiều trách nhiệm, việc thực hiện kiểm thử sẽ trở nên khó khăn.
- **Làm việc với Phát triển dựa trên kiểm thử (TDD - Test Driven Development):** Trong TDD, các trường hợp kiểm thử được viết trước khi bắt đầu phát triển tính năng. Trong TDD, các nhà phát triển chỉ viết mã khi các bài kiểm thử tự động không thành công. Nó cũng có những lợi ích nhất định, chẳng hạn như ít lỗi hơn và tái cấu trúc mã dễ dàng hơn.
- **Thực hiện kiểm thử hồi quy:** Bất cứ khi nào một tính năng mới được giới thiệu hoặc các bản cập nhật được thực hiện, nó không được phá vỡ sự ổn định hoặc gây ra bất kỳ lỗi nào trong cơ sở mã hiện có. Vì vậy, các bài kiểm thử hồi quy được thực hiện để đảm bảo rằng bất kỳ thay đổi nào được thực hiện không ảnh hưởng đến sự ổn định của mã.
- **Tích hợp các bài kiểm thử với CI/CD:** Điều này giúp tự động hóa các trường hợp kiểm thử. Ví dụ, khi bất kỳ mã nào được đẩy vào một kho lưu trữ từ xa, các trường hợp kiểm thử tự động sẽ được kích hoạt để thực thi và chuẩn bị báo cáo kiểm thử. Tự động hóa các bài kiểm thử với CI/CD giúp giảm thiểu các lỗi thủ công.
- **Độ bao phủ kiểm thử tối đa:** Bạn nên viết tất cả các loại trường hợp kiểm thử để bao phủ tối đa. Ví dụ, viết các trường hợp kiểm thử bao gồm cả các tình huống mong đợi và không mong đợi giúp chúng ta bao phủ tối đa.

Đây là một số phương pháp tốt nhất cần được tuân thủ trong khi kiểm thử. Ngoài ra, các tổ chức cũng nên sử dụng một môi trường kiểm thử chuyên dụng thay vì dựa vào đám mây vì các lỗ hổng bảo mật như rò rỉ dữ liệu riêng tư của tổ chức, các cuộc tấn công mạng, v.v., liên quan đến đám mây.

Session 4: Xử Lý Ngoại Lệ và Ghi Log

Tác giả: Trần Hoàng Sơn

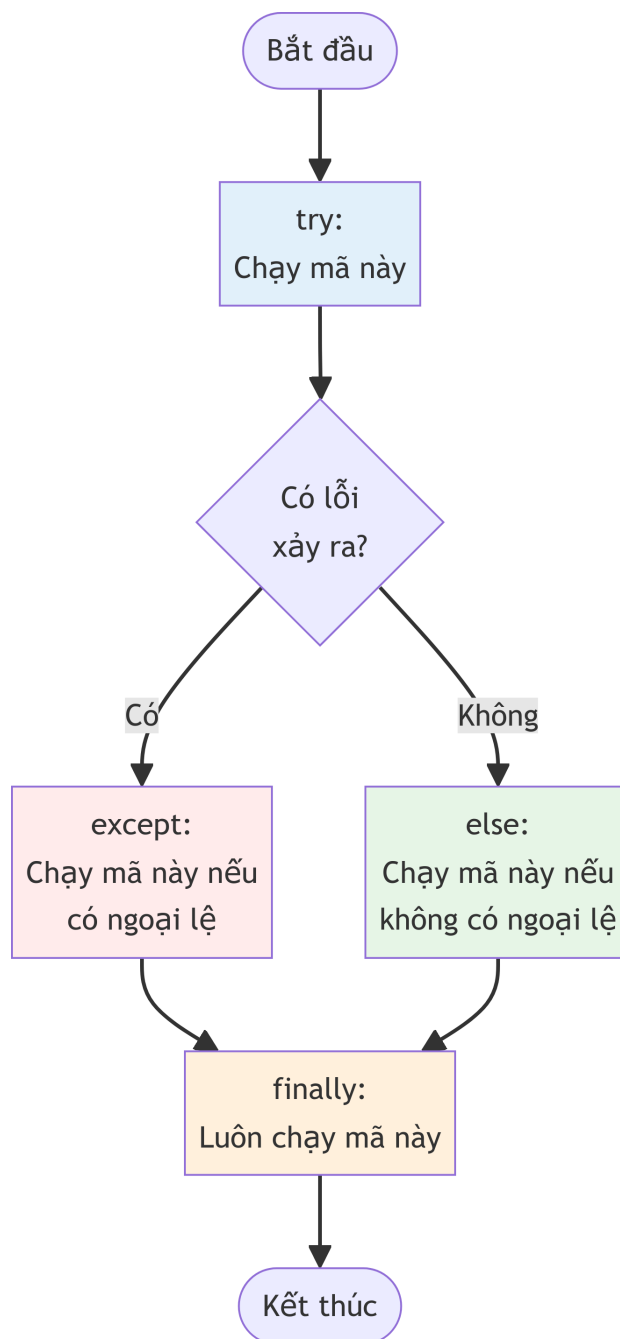
4.1 Hiểu Về Xử Lý Ngoại Lệ (Exception Handling)

Xử lý ngoại lệ là quá trình xử lý các điều kiện đặc biệt xảy ra trong quá trình chạy (runtime) để chương trình/phần mềm không bị sập và tiếp tục hoạt động. Ví dụ về các trường hợp cần xử lý ngoại lệ là không đủ bộ nhớ để chạy chương trình, các vấn đề kết nối cơ sở dữ liệu có thể xảy ra do mất điện hoặc máy chủ không thể truy cập, đầu vào không hợp lệ do người dùng nhập, v.v.

Python là một ngôn ngữ thông dịch (interpreter-based) và không sử dụng trình biên dịch (compiler); do đó, việc xử lý ngoại lệ trở nên bắt buộc. Ở đây, các chỉ thị được chuyển đổi thành mã máy và thực thi lần lượt. Mặt khác, trong ngôn ngữ dựa trên trình biên dịch, toàn bộ mã nguồn trước tiên được chuyển đổi thành mã máy và sau đó được thực thi cùng một lúc. Ví dụ, C++, Java, v.v. Trong các ngôn ngữ dựa trên trình biên dịch, vì tất cả mã được chuyển đổi thành ngôn ngữ máy trước khi thực thi, nhiều lỗi được xác định ở giai đoạn biên dịch, điều này không xảy ra trong các ngôn ngữ thông dịch như Python.

Trong Python, một số từ khóa được xác định trước được sử dụng để xử lý các ngoại lệ. Chúng như sau:

- **try:** Trong khối `try`, bạn viết mã cho tác vụ cần thực hiện, nơi các lỗi có thể xảy ra trong quá trình chạy.
- **except:** Trong khối `except`, bạn viết các tác vụ cần thực hiện trong trường hợp có bất kỳ ngoại lệ nào xảy ra.
- **else:** Mã được viết trong khối `else` chỉ được thực thi khi không có ngoại lệ nào xảy ra. Nói một cách đơn giản, khối `except` hoặc khối `else` sẽ được thực thi.



Lưu ý: Ngoài các từ khóa `try`, `except` và `else`, còn có một từ khóa nữa được gọi là `finally`. Mã được viết trong khối `finally` sẽ luôn được thực thi, bất kể có bất kỳ ngoại lệ nào xảy ra hay không. Ví dụ, nếu một tệp được mở, nó phải được đóng trong mọi điều kiện, bất kể có bất kỳ ngoại lệ nào xảy ra hay không. Loại mã như vậy có thể được viết trong khối `finally`.

Mã có thể gây ra lỗi được viết bên trong khối `try` và các trình xử lý ngoại lệ khác nhau xử lý các ngoại lệ khác nhau, như `ZeroDivisionError` và `ValueError`, bên trong khối `except`. Ngoài ra, một trình xử lý ngoại lệ chung cũng có thể được sử dụng, được gọi là `Exception`. Trình xử lý này có thể xử lý bất kỳ loại ngoại lệ nào. Do đó, nên đặt nó sau

khi xác định tất cả các khối ngoại lệ khác. Nếu bạn xác định trình xử lý ngoại lệ chung này ở giữa, nó sẽ không để ngoại lệ đến được các khối tương ứng của chúng. Cú pháp chung để xác định một trình xử lý ngoại lệ như sau:

```
except <Loại_Trình_Xử_Lý> as <tên_đối_tượng>
```

Ở đây, `Loại_Trình_Xử_Lý` là loại ngoại lệ bạn muốn xử lý. Python hỗ trợ rất nhiều ngoại lệ sẵn có. Sau đó, `tên_đối_tượng` là biến mà bạn muốn lưu trữ đối tượng ngoại lệ để truy cập sau này.

Bây giờ, hãy tóm tắt các ưu điểm của việc xử lý ngoại lệ:

Ưu Điểm Của Exception Handling

- ✓ Xử lý lỗi runtime để ứng dụng không bị crash
- ✓ Tách biệt mã xử lý lỗi khỏi logic chính
- ✓ Nhóm các lỗi dựa trên các exception blocks khác nhau

4.2 Ghi Log trong Python (Logging)

Trong các kịch bản thực tế, bạn cần theo dõi mọi vấn đề nhỏ hoặc lỗi xảy ra trong ứng dụng. Đây là lúc ghi log cực kỳ hữu ích. Có nhiều công cụ ghi log tiên tiến có sẵn trên thị trường như ELK Stack, Datadog, v.v. Python cũng có hỗ trợ sẵn có cho việc ghi log.

Lưu ý: Câu lệnh `print` phù hợp với một số mã chung bạn viết trên máy của mình. Nhưng khi nói đến môi trường cấp sản xuất, lỗi có thể xảy ra bất cứ lúc nào, nơi các câu lệnh `print` có thể không hữu ích vì bạn sẽ không thể theo dõi chúng. Do đó, các tệp log có thể được tạo để lưu trữ nhật ký của mọi phiên bản. Sau đó, các nhà phát triển có thể sử dụng chúng để gỡ lỗi sự cố.

Trong Python, chúng ta sử dụng mô-đun `logging` để tạo các tệp log và ghi vào chúng. Có năm cấp độ ghi log khác nhau trong python, mỗi cấp độ có một mức độ nghiêm trọng khác nhau. Chúng như sau:

Các Mức Độ Logging trong Python

	DEBUG (10)	Chẩn đoán vấn đề, debug phần mềm
	INFO (20)	Xác nhận mọi thứ hoạt động bình thường
	WARNING (30)	Cảnh báo vấn đề có thể xảy ra
	ERROR (40)	Lỗi nghiêm trọng, tính năng không hoạt động
	CRITICAL (50)	Lỗi cực kỳ nghiêm trọng, hệ thống có thể sập

- **debug:** Cấp độ này được sử dụng khi bạn đang chẩn đoán một số vấn đề. Mức độ nghiêm trọng cho `debug` là 10. Nó hữu ích khi bạn gỡ lỗi phần mềm của mình để đảm bảo mọi thứ đang chạy tốt.
- **info:** Cấp độ này được sử dụng để thông báo cho bạn rằng một sự kiện cụ thể nào đó đã xảy ra, chẳng hạn như ứng dụng của bạn đã vào một trạng thái nhất định, v.v. Nói một cách đơn giản, nó cung cấp cho bạn xác nhận rằng mọi thứ đang hoạt động tốt. Mức độ nghiêm trọng cho `info` là 20.
- **warning:** Cấp độ này cho biết rằng một điều gì đó không mong muốn đã xảy ra hoặc một vấn đề sẽ phát sinh trong tương lai. Tuy nhiên, ứng dụng vẫn đang hoạt động. Ví dụ, đột nhiên, số lượng người dùng trang web đã tăng lên, liên tục ảnh hưởng đến thời gian phản hồi cho một tính năng cụ thể. Sau đó, nó thuộc cấp độ cảnh báo. Mức độ nghiêm trọng cho `warning` là 30.
- **error:** Cấp độ này cho biết rằng một số tính năng hoặc chức năng không hoạt động và cần được khắc phục. Mức độ nghiêm trọng cho `error` là 40. Ví dụ, giả sử tính năng được thêm gần đây vào trang web không hoạt động bình thường và cũng ảnh hưởng đến các tính năng khác.
- **critical:** Cấp độ này cho biết rằng một số lỗi nghiêm trọng đã xảy ra cần được kiểm tra càng sớm càng tốt. Ví dụ, giả sử một vấn đề với một số tính năng đã bị đóng băng hoặc làm sập toàn bộ trang web. Mức độ nghiêm trọng cho `critical` là cao nhất, tức là 50.

Có một số hàm/phương thức quan trọng được sử dụng để thực hiện các cài đặt cấu hình ban đầu. Chúng như sau:

- **basicConfig:** Phương thức này cấu hình một số cài đặt cấp ban đầu trước khi đối tượng logger của bạn có thể bắt đầu ghi log các thông báo. Nó có nhiều tham số khác nhau, như sau:
 - **filename:** Theo mặc định, tất cả các thông báo log được in trên terminal/console. Nhưng nếu bạn muốn chúng được lưu trong một tệp, bạn có thể chỉ định chúng ở đây.
 - **format:** Tham số này được sử dụng để chỉ định cấu trúc của thông báo log. Ví dụ, trong trường hợp đã cho, `%(asctime)s%(message)s` , `%(asctime)s` được sử dụng cho ngày và giờ, và `%(message)s` được sử dụng cho thông báo thực tế cần được ghi log. Bạn có thể tìm thấy các định dạng hoàn chỉnh có sẵn trong phần thuộc tính LogRecord.
 - **filemode:** Tham số này được sử dụng để chỉ định chế độ mà bạn muốn mở tệp. Ví dụ, 'a' là viết tắt của chế độ nối thêm (append).
- **getLogger():** Phương thức này được sử dụng để tạo một đối tượng logger. Ví dụ, `logger = logging.getLogger()` sẽ tạo một đối tượng logger và lưu trữ tham chiếu trong đối tượng logger. Bây giờ, bất kỳ thông báo log nào được lưu trữ sẽ chỉ được ghi log bởi logger này.

Lưu ý: Nếu bạn không tạo bất kỳ đối tượng logger nào, thì theo mặc định, root logger được sử dụng làm logger.

- **setLevel():** Phương thức này được sử dụng để đặt mức ngưỡng của logger. Ví dụ, `logger.setLevel(logging.DEBUG)` sẽ đặt mức ngưỡng thành `DEBUG` , có nghĩa là chỉ các thông báo có mức độ nghiêm trọng bằng hoặc lớn hơn `DEBUG` mới được ghi log trong console hoặc tệp. Bây giờ, giả sử mức được đặt là `WARNING(30)` , thì chỉ các thông báo `WARNING(30)` , `ERROR(40)` và `CRITICAL(50)` mới được ghi log. `DEBUG(10)` và `INFO(20)` sẽ bị bỏ qua vì chúng có mức độ nghiêm trọng thấp hơn so với `WARNING` .

Session 5: Chuyển Đổi Từ Notebook Sang Scripts

Tác giả: Trần Hoàng Sơn

Trong session này, bài toán được đặt ra là xây dựng một hệ thống dựa trên ML có thể phân loại xem một người có bị bệnh tim hay không dựa trên các thông số nhất định.

Bạn bắt đầu bằng cách huấn luyện các mô hình ML khác nhau trong một jupyter notebook. Sau khi hoàn thiện mô hình của mình, bạn đã tạo một notebook suy luận (inference notebook), được sử dụng để lấy dự đoán của mô hình trên dữ liệu trực tiếp. Nhớ lại rằng bạn đã sử dụng các bước tiền xử lý dữ liệu tương tự như khi chúng ta đã làm trong quá trình huấn luyện cho quá trình suy luận.

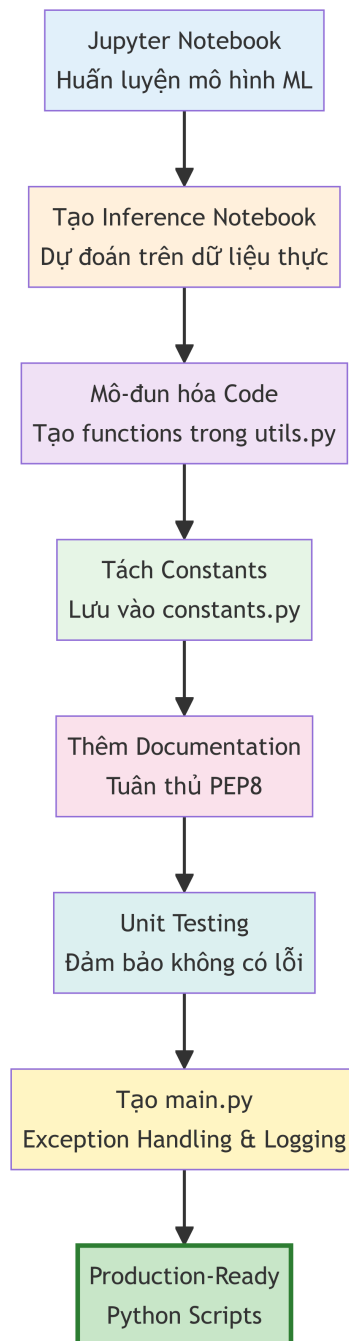
Sau khi tạo thành công notebook suy luận, bạn đã mô-đun hóa mã của mình bằng cách trừu tượng hóa mã thành một hàm. Bạn đã lưu trữ tất cả các hàm tiện ích trong tệp `utils.py` và lưu trữ tất cả các hằng số trong tệp `constants.py` để chúng ta có thể dễ dàng cấu hình các hằng số của mình sau này.

Sau khi hoàn thành việc này, bạn đã thêm tài liệu vào mã của mình và đảm bảo rằng nó tuân thủ các nguyên tắc pep8. Điều này đảm bảo rằng mã của bạn có thể đọc được bởi những người khác, do đó tạo điều kiện cho sự hợp tác suôn sẻ.

Sau đó, bạn đã thực hiện kiểm thử đơn vị (unit testing) để đảm bảo rằng mã hoạt động như dự định và không có lỗi nào trong mã.

Khi bạn chắc chắn rằng không có lỗi nào trong mã của mình, bạn đã tạo tệp `main.py` và bao gồm xử lý ngoại lệ và ghi log.

Với điều này, bạn đã có thể tạo một mã quy trình suy luận sẵn sàng cho sản xuất.



Bây giờ chúng ta hãy hiểu tại sao các tập lệnh python được ưu tiên hơn so với Jupyter notebook trong các Hệ thống ML sản xuất.

Tại Sao Python Scripts Tốt Hơn Jupyter Notebook?

Vấn Đề Với Jupyter Notebook:

- X Khó quản lý phiên bản code và theo dõi thay đổi
- X Khó thiết lập dependency management
- X Khó peer review và làm việc nhóm
- X Thách thức trong tự động hóa CI/CD

Ưu Điểm Của Python Scripts:

- ✓ Dễ dàng version control với Git
- ✓ Quản lý dependencies rõ ràng (requirements.txt)
- ✓ Hỗ trợ collaboration và code review tốt
- ✓ Tích hợp CI/CD pipeline dễ dàng

- Khó thực hiện quản lý phiên bản mã phù hợp và theo dõi các thay đổi
- Khó thiết lập quản lý phụ thuộc cho notebook
- Khó khăn cho việc đánh giá ngang hàng và làm việc trên cùng một notebook bởi nhiều nhà phát triển
- Những thách thức trong việc tự động hóa tích hợp liên tục và triển khai liên tục mã (CI/CD)

Cấu Trúc Project Production-Ready

📁 project/

├── 📄 main.py # Entry point, xử lý lỗi & logging

├── 📄 utils.py # Các hàm tiện ích

├── 📄 constants.py # Các hằng số cấu hình

├── 📄 test_utils.py # Unit tests

├── 📄 requirements.txt # Dependencies

└── 📁 models/ # Lưu trữ mô hình ML